

Client-Side Testing Vitest, Playwright E2E, Babylon.js NullEngine & Performance

Author: Daniel | **Date:** Friday, 18 September 2026 | **Time:** 16:39 CDT |

Location: Anoka, MN

Series: HavenWorld Build Guide | **Part:** 7B | **Status:** Active Development

Tech Stack: Babylon.js 7.x · Vite 6 · TypeScript · Vitest 2.x · Playwright 1.50+ · Socket.io 4 · Express 5

Deployment: Cloudflare Pages — <https://havenworld.pages.dev>

Series Context

Part 7A covered server-side testing (API, Socket.io, database). This document, Part 7B, covers all client-side testing exclusively. All commands are tagged **MAC** (MacBook Pro dev machine) unless a **LINUX** tag appears. Part 8 will cover CI/CD pipeline hardening and deployment automation.

Table of Contents

1. Introduction — Why Client Testing Is Different
2. Setting Up Vitest
3. Setting Up Playwright
4. Unit Tests — AuthService
5. Unit Tests — SocketService

6. Unit Tests — AvatarController
 7. Unit Tests — RoomLoader
 8. Unit Tests — FurnitureManager
 9. Unit Tests — AudioEngine
 10. Unit Tests — ChatOverlay
 11. Babylon.js Scene Tests — NullEngine
 12. Playwright E2E — Full Game Flow
 13. Playwright E2E — Two-Player Interaction
 14. Playwright E2E — Avatar Customization
 15. Visual Regression Testing
 16. Performance Testing — Playwright CDP
 17. Accessibility Testing
 18. Mobile Emulation Testing
 19. Test Data Management
 20. CI Integration — GitHub Actions
 21. Client Test Coverage Checklist
-

1. Introduction — Why Client Testing Is Different

Server-side testing (covered in Part 7A) is comparatively straightforward: your code runs in a single Node.js environment with deterministic I/O, no rendering surface, and no human-timing dependencies. Client-side testing for a browser-based game like HavenWorld introduces an entirely different class of challenge. The client must coordinate a real-time rendering engine (Babylon.js), live WebSocket events (Socket.io), DOM-based UI overlays (login form, chat, shop), user input (mouse clicks, keyboard, touch), and browser APIs (Web Audio, WebGL, localStorage) — often simultaneously.

The core complications unique to the HavenWorld client are: (1) **WebGL** — Babylon.js uses GPU-backed rendering that cannot run in a standard Node.js test

environment; (2) **network timing** — socket events arrive asynchronously and must be mocked with deterministic timing in unit tests; (3) **user interactions** — pointer events, touch, keyboard, and gamepad input require a real or emulated browser; and (4) **stateful animations** — avatar animation state machines advance frame-by-frame and must be driven by a fake clock in isolation.

HavenWorld solves these by layering four distinct testing tiers, each with a specific tool and scope. Tests at lower layers run faster and catch regressions sooner; tests at higher layers validate integration and real-world behavior. All four tiers are required — no single tier is sufficient on its own.

1.1 The HavenWorld Client Testing Pyramid

Tier	Type	Tool	Scope	Speed	Count Target
1 — Unit	Pure logic	Vitest 2.x	Pure functions, service classes, state machines, JWT parsing	< 5s total	~120 tests
2 — Component	UI behavior	Vitest + jsdom + @testing-library/dom	Login form, chat overlay, shop modal, avatar customizer panel	< 15s total	~60 tests
3 — Scene	3D logic	Vitest + Babylon.js NullEngine	Scene graph, animations, lighting, physics, navmesh, pet FSM	< 30s total	~45 tests
4 — E2E	Full browser	Playwright 1.50+	Login→room→move→chat→logout, multiplayer, avatar customization, accessibility, mobile, performance	3-8 min total	~35 tests

1.2 Tools Summary

Tool	Version	Role	Why This Over Alternatives
Vitest	2.x	Unit & component runner	Vite-native; shares vite.config.ts aliases and plugins; ~3× faster HMR than Jest for TS browser code
jsdom	bundled	DOM environment for Vitest	Headless DOM for UI component tests without launching a browser
@testing-library/dom	latest	DOM query helpers	Semantic queries (getByRole, getByLabel) that mirror real user behavior
Babylon.js NullEngine	7.x (bundled)	Headless 3D rendering	Runs all Babylon scene logic, animations, physics — without any GPU or WebGL context
Playwright	1.50+	E2E browser automation	Built-in Chromium/Firefox/WebKit; device emulation; CDP access for performance metrics; parallel contexts
@vitest/coverage-v8	2.x	Coverage instrumentation	Uses V8 native coverage (faster than Istanbul); works with Vite transforms

Tip: Why Not Jest?

Jest requires separate babel/ts-jest transforms and cannot reuse your Vite plugin config. For HavenWorld's Babylon.js imports (which are large ESM modules), Vitest's native ESM support cuts cold-start time from ~12 seconds (Jest) to ~3 seconds. Use Vitest for everything in the `src/client/` tree; use Jest only if you

inherit a legacy server test suite from Part 7A.

2. Setting Up Vitest MAC

2.1 Install Dependencies

Run from the project root. These are all dev dependencies — none ship to production:

```
# [MAC] From project root — installs all Vitest deps at once npm install --save-dev \ vitest@^2.0.0 \ @vitest/coverage-v8@^2.0.0 \ jsdom \ @testing-library/dom \ @testing-library/user-event
```

Tip: Version Pinning

Pin vitest and @vitest/coverage-v8 to the same minor version (both ^2.0.0). A mismatch between these two packages is the single most common cause of Vitest startup failures. Always update them together: `npm install --save-dev vitest@latest @vitest/coverage-v8@latest`.

2.2 vitest.config.ts

Create `vitest.config.ts` at the project root. This file is separate from `vite.config.ts` but deliberately imports from it so that path aliases and plugins are shared. Without this, Vitest would not resolve `@/client/...` aliases the same way Vite does during the actual build — causing tests to pass locally but fail to find modules in CI.

```
// vitest.config.ts import { defineConfig, mergeConfig } from 'vitest/config'; import viteConfig from './vite.config'; export default mergeConfig(viteConfig, defineConfig({ test: { // jsdom gives us a DOM environment for UI component tests environment: 'jsdom', // Enable describe/it/expect without imports in every file globals: true, // Only scan client-side test files include: [ 'src/client/**/__tests__/**/*.{test,spec}.{ts,tsx}', '__tests__/scene/**/*.{test,spec}.{ts,tsx}', ], // Run setup file before each test suite (resets DOM, mocks) setupFiles: 
```

```
[ './src/client/__tests__/setup.ts'], // Load test-specific env vars
(.env.test overrides .env) env: { VITE_API_URL:
'http://localhost:4000', }, coverage: { provider: 'v8',
reporter: ['text', 'html', 'lcov'], reportsDirectory:
'./coverage/client', include: ['src/client/**/*.ts,tsx'],
exclude: [ 'src/client/**/*.__tests__/**',
'src/client/**/*.d.ts', 'src/client/main.ts', // entry
point, not unit-testable ], thresholds: { // 60%
floor - raise to 75% after suite is mature lines: 60,
functions: 60, branches: 60, statements: 60, },
}, }, }) );
```

2.3 vite.config.ts Integration

Your main `vite.config.ts` should include a `test` field with shared configuration. The key point is that Vitest reads from `vite.config.ts` first, then merges with `vitest.config.ts`. Keeping aliases in `vite.config.ts` ensures the build and the tests always resolve modules identically:

```
// vite.config.ts (relevant sections - add to existing config) import
{ defineConfig } from 'vite'; import path from 'path'; export default
defineConfig({ resolve: { alias: { // @/ resolves to src/client/
in both build AND tests '@': path.resolve(__dirname, 'src/client'),
// Prevents Babylon.js from pulling in full ES module side effects
'@babel/core': '@babel/core/index.js', }, }, // Vitest picks
this up via mergeConfig - do not duplicate in vitest.config.ts test:
{ globals: true, }, });
```

2.4 tsconfig.test.json

TypeScript needs a separate config for test files so that Vitest's global types (`describe`, `it`, `expect`) are available, and so test files can use DOM types alongside Node types without polluting the production build:

```
// tsconfig.test.json - extends base client TS config { "extends":
"./tsconfig.client.json", "compilerOptions": { "types":
["vitest/globals", "node", "@testing-library/dom"], "lib": ["ES2022",
"DOM", "DOM.Iterable"], "noEmit": true, "isolatedModules": true },
"include": [ "src/client/**/*.__tests__/**/*.ts",
"__tests__/scene/**/*.ts", "e2e/**/*.ts" ] }
```

2.5 package.json Scripts

```
// package.json - add to "scripts" block { "scripts": { // Run all
client unit+component+scene tests once (used in CI) "test:client":
"vitest run --config vitest.config.ts", // Watch mode for local
development - re-runs affected tests on save "test:client:watch": "vitest
--config vitest.config.ts", // Coverage report - generates HTML report
in ./coverage/client/ "test:client:coverage": "vitest run --config
vitest.config.ts --coverage", // Run only scene tests (NullEngine -
slower, run separately) "test:client:scene": "vitest run --config
vitest.config.ts --reporter=verbose __tests__/scene" } }
```

2.6 .env.test for Client

```
# .env.test – loaded by Vitest automatically (gitignore this in real projects)
VITE_API_URL=http://localhost:4000
VITE_SOCKET_URL=http://localhost:4000 VITE_ENV=test TEST_INVITE_CODE=HAVEN-TEST-2026
TEST_API_URL=http://localhost:4000
```

2.7 Test Setup File

The global setup file runs before every test suite. It resets the DOM, restores all mocks, and stubs browser APIs that jsdom does not implement (ResizeObserver, matchMedia). Without this, tests become stateful and can fail in unpredictable order:

```
// src/client/__tests__/setup.ts
import { afterEach, vi } from 'vitest';
import { cleanup } from '@testing-library/dom'; // Clean up any DOM nodes rendered during the previous test
afterEach(() => { cleanup(); });
vi.restoreAllMocks(); // jsdom does not implement ResizeObserver – stub it
global.ResizeObserver = vi.fn().mockImplementation(() => ({ observe: vi.fn(), unobserve: vi.fn(), disconnect: vi.fn(), })); // jsdom doesn't support matchMedia – stub for responsive UI tests
Object.defineProperty(window, 'matchMedia', { writable: true, value: vi.fn().mockImplementation((query: string) => ({ matches: false, media: query, onchange: null, addListener: vi.fn(), removeListener: vi.fn(), addEventListener: vi.fn(), removeEventListener: vi.fn(), dispatchEvent: vi.fn(), })), });
```

2.8 Recommended File Structure

```
src/  client/  services/  authService.ts  socketService.ts
__tests__/    authService.test.ts  socketService.test.ts
game/         avatarController.ts  roomLoader.ts  furnitureManager.ts
__tests__/    avatarController.test.ts  roomLoader.test.ts
furnitureManager.test.ts  ui/  chatOverlay.ts  shopModal.ts
__tests__/    chatOverlay.test.ts  shopModal.test.ts
__tests__/    setup.ts  # global test setup __tests__/
scene/        roomScene.test.ts  # NullEngine scene tests e2e/
fullFlow.spec.ts  multiplayer.spec.ts  avatarCustomizer.spec.ts  visual/
rooms.spec.ts  performance/  perf.spec.ts  accessibility/
a11y.spec.ts  mobile/  mobile.spec.ts  pages/  LoginPage.ts
# Page Object Model classes  LobbyPage.ts  RoomPage.ts
```

3. Setting Up Playwright MAC

3.1 Install Playwright

```
# [MAC] Install Playwright test runner npm install --save-dev @playwright/test
# [MAC] Download all three browser engines (Chromium,
```

```
Firefox, WebKit) # This installs ~300MB of browser binaries under
~/.cache/ms-playwright/ npx playwright install # [MAC] Optional: install
system deps needed by browsers (usually only needed on Linux CI) npx
playwright install-deps
```

Tip: Chromium vs Chrome

npx playwright install installs Playwright's own pinned Chromium build — NOT your system Chrome. This is intentional: it guarantees deterministic rendering across all developer machines and CI. Never point Playwright at `/Applications/Google Chrome.app` in your config, as Chrome updates independently and can silently break visual regression baselines.

3.2 playwright.config.ts

```
// playwright.config.ts import { defineConfig, devices } from
'@playwright/test'; import path from 'path'; export default defineConfig({
// Directory where all .spec.ts E2E files live testDir: './e2e', // Base
URL – Vite dev server started by globalSetup use: { baseUrl:
'http://localhost:5173', // Capture screenshot + trace on failure for
debugging screenshot: 'only-on-failure', trace: 'retain-on-failure',
video: 'retain-on-failure', }, // Retry once on CI (flakiness from
network timing), 0 locally retries: process.env.CI ? 1 : 0, // Run test
files in parallel (each gets its own browser context) fullyParallel: true,
workers: process.env.CI ? 2 : 4, // 30s per test – Babylon.js room loads
can be slow under load timeout: 30_000, reporter: [ ['list'],
['html', { open: 'never', outputFolder: 'playwright-report' }], ], //
Start Vite dev server + local backend before tests run globalSetup:
'./e2e/globalSetup.ts', globalTeardown: './e2e/globalTeardown.ts',
projects: [ { name: 'chromium', use: { ...devices['Desktop
Chrome'] }, }, { name: 'firefox', use:
{ ...devices['Desktop Firefox'] }, }, { name: 'webkit',
use: { ...devices['Desktop Safari'] }, }, // Mobile projects (nightly
only – see CI section) { name: 'mobile-chrome', use:
{ ...devices['Pixel 7'] }, testMatch: '**/mobile/**/*spec.ts', },
{ name: 'mobile-safari', use: { ...devices['iPhone 14'] },
testMatch: '**/mobile/**/*spec.ts', }, ], });
```

3.3 Global Setup — Start Vite + Backend

```
// e2e/globalSetup.ts import { chromium, FullConfig } from
'@playwright/test'; import { createServer } from 'vite'; import { spawn,
ChildProcess } from 'child_process'; let viteServer:
Awaited<ReturnType<typeof createServer>>; let backendProcess: ChildProcess;
export default async function globalSetup(config: FullConfig) { // Start
the Vite dev server programmatically viteServer = await
createServer({ configFile: './vite.config.ts', server: { port:
5173 }, mode: 'test', }); await viteServer.listen();
console.log('[E2E Setup] Vite dev server started on :5173'); // Start
```

```

local backend (Express + Socket.io) as a child process  backendProcess =
spawn('node', ['dist/server/index.js'], {    env: { ...process.env,
NODE_ENV: 'test', PORT: '4000' },    stdio: 'inherit',    });    // Wait for
backend to be ready before tests run    await waitForPort(4000, 10_000);
console.log('[E2E Setup] Backend ready on :4000');    // Save authenticated
session state for tests that skip login    await seedAuthState(config); }
async function waitForPort(port: number, timeout: number): Promise<void>
{    const start = Date.now();    while (Date.now() - start < timeout)
{        try {            await fetch(`http://localhost:${port}/health`);
return;        } catch {            await new Promise(r => setTimeout(r, 500));        }
    }    throw new Error(`Port ${port} did not become ready within ${timeout}ms`);
}    async function seedAuthState(config: FullConfig): Promise<void> {    // Log
in once and save storageState for tests that need an authenticated user
const browser = await chromium.launch();    const page = await
browser.newPage();    await page.goto('http://localhost:5173/login');    await
page.fill('[name=email]', 'testuser_a@havenworld.test');    await
page.fill('[name=password]', process.env.TEST_USER_A_PASS!);    await
page.click('button[type=submit]');    await page.waitForURL('**/lobby');
await page.context().storageState({ path: 'e2e/.auth/user_a.json' });    await
browser.close(); }

```

3.4 package.json Scripts

```

// Add to package.json "scripts" {    "scripts": {        // Run all E2E tests
headlessly (CI default)        "test:e2e": "playwright test",        // Run E2E
with browser windows visible – useful for debugging        "test:e2e:headed":
"playwright test --headed",        // Open Playwright Inspector on a specific
test        "test:e2e:debug": "playwright test --debug",        // Run only
performance tests (slow, not in default CI run)        "test:perf": "playwright
test e2e/performance/ --project=chromium",        // Run only mobile tests
"test:mobile": "playwright test e2e/mobile/ --project=mobile-chrome --
project=mobile-safari",        // Open Playwright HTML report after a failed
run        "test:e2e:report": "playwright show-report playwright-report"    } }

```

3.5 Page Object Model Classes

Page Object Model (POM) encapsulates all selectors and interactions for a page into a reusable class. This means when HavenWorld's DOM changes, you update one class rather than hunting through every test file. POM is not optional for a project of this scale — without it, E2E tests become unmaintainable within months.

```

// e2e/pages/LoginPage.ts import { Page, Locator, expect } from
'@playwright/test'; export class LoginPage {    readonly page: Page;
readonly emailInput: Locator;    readonly passwordInput: Locator;    readonly
submitButton: Locator;    readonly errorMessage: Locator;    readonly
registerLink: Locator;    constructor(page: Page) {        this.page = page;
this.emailInput = page.locator('[name=email]');        this.passwordInput =
page.locator('[name=password]');        this.submitButton =
page.locator('button[type=submit]');        this.errorMessage =
page.locator('[data-testid=auth-error]');        this.registerLink =
page.locator('[data-testid=register-link]');    }    async goto() {        await
this.page.goto('/login');        await
expect(this.emailInput).toBeVisible();    }    async login(email: string,
password: string) {        await this.emailInput.fill(email);        await
this.passwordInput.fill(password);        await this.submitButton.click();    } }

```

```
// e2e/pages/LobbyPage.ts import { Page, Locator, expect } from
'@playwright/test'; export class LobbyPage { readonly page: Page;
readonly avatarCanvas: Locator; readonly roomList: Locator; readonly
havenParkButton: Locator; readonly logoutButton: Locator;
constructor(page: Page) { this.page = page; this.avatarCanvas =
page.locator('canvas[data-testid=avatar-preview]'); this.roomList =
page.locator('[data-testid=room-list]'); this.havenParkButton =
page.locator('[data-testid=room-haven-park]'); this.logoutButton =
page.locator('[data-testid=logout-btn]'); } async waitForLoad()
{ await expect(this.avatarCanvas).toBeVisible({ timeout: 10_000 });
await expect(this.roomList).toBeVisible(); } } // e2e/pages/RoomPage.ts
import { Page, Locator, expect } from '@playwright/test'; export class
RoomPage { readonly page: Page; readonly gameCanvas: Locator; readonly
chatInput: Locator; readonly chatMessages: Locator; readonly chatToggle:
Locator; constructor(page: Page) { this.page = page;
this.gameCanvas = page.locator('canvas[data-testid=game-canvas]');
this.chatInput = page.locator('[aria-label="Chat message"]');
this.chatMessages = page.locator('[data-testid=chat-messages]');
this.chatToggle = page.locator('[data-testid=chat-toggle]'); } async
waitForRoomLoad() { // Wait for canvas to appear AND for the room-ready
event await expect(this.gameCanvas).toBeVisible({ timeout: 15_000 });
await this.page.waitForFunction(() => (window as any).__havenRoomReady
=== true, { timeout: 15_000 } ); } async clickCanvas(x: number, y:
number){ await this.gameCanvas.click({ position: { x, y } }); }
async sendMessage(text: string) { await this.chatInput.click();
await this.chatInput.fill(text); await this.chatInput.press('Enter'); }
}
```

4. Unit Tests — AuthService MAC

AuthService is the most security-critical client-side module. It is responsible for storing the access token exclusively in JavaScript memory (never `localStorage` or `sessionStorage`, which are XSS-accessible), handling silent token refresh via an `httpOnly` refresh cookie, and exposing `isAuthenticated()` with expiry-awareness. Every edge case tested here corresponds to a real security or session-management failure mode.

File: `src/client/services/__tests__/authService.test.ts`

```
// src/client/services/__tests__/authService.test.ts import { describe, it,
expect, vi, beforeEach, afterEach } from 'vitest'; import { AuthService,
AuthError } from '../authService'; // Helper: build a JWT with a given exp
claim (iat is now, exp is now + offsetSeconds) function
makeJwt(offsetSeconds: number): string { const payload = { sub:
'user_123', iat: Math.floor(Date.now() / 1000), exp:
Math.floor(Date.now() / 1000) + offsetSeconds, }; const encoded =
btoa(JSON.stringify(payload)); return `header.${encoded}.signature`; }
describe('AuthService', () => { let service: AuthService; beforeEach(()
=> { service = new AuthService(); // Reset all fetch mocks before
every test vi.resetAllMocks(); }); // — Test 4a
it('(a) login()
```

```

success → token stored in memory, NOT in localStorage', async () =>
{
  const token = makeJwt(3600);
  global.fetch =
vi.fn().mockResolvedValueOnce({ ok: true, json: async () =>
({ accessToken: token }), } as Response);
  await
service.login('user@havenworld.com', 'password123'); // Token MUST be in
memory expect(service.getToken()).toBe(token); // Token MUST NOT be
in localStorage (XSS protection)
expect(localStorage.getItem('accessToken')).toBeNull();
expect(sessionStorage.getItem('accessToken')).toBeNull(); }); // — Test
4b
it('(b) login()
failure → throws AuthError, no token stored', async () => {
  global.fetch
= vi.fn().mockResolvedValueOnce({ ok: false, status: 401,
json: async () => ({ message: 'Invalid credentials' }), } as Response);
  await expect(service.login('bad@email.com',
'wrong')).rejects.toThrow(AuthError);
expect(service.getToken()).toBeNull(); }); // — Test 4c
it('(c) getToken()
when logged in → returns current token', async () => {
  const token =
makeJwt(3600);
  global.fetch = vi.fn().mockResolvedValueOnce({ ok:
true, json: async () => ({ accessToken: token }), } as Response);
  await service.login('user@havenworld.com', 'pass');
expect(service.getToken()).toBe(token); }); // — Test 4d
it('(d) getToken()
when expired → triggers silent refresh, returns new token', async () => {
  const expiredToken = makeJwt(-10); // already expired 10s ago
  const
freshToken = makeJwt(3600);
  global.fetch = vi.fn() // First call:
login with soon-to-expire token .mockResolvedValueOnce({ ok: true,
json: async () => ({ accessToken: expiredToken }) } as Response) //
Second call: silent refresh returns a fresh
token .mockResolvedValueOnce({ ok: true, json: async () =>
({ accessToken: freshToken }) } as Response);
  await
service.login('user@havenworld.com', 'pass'); // getToken should detect
expiry, refresh silently, then return new token
  const result = await
service.getToken();
  expect(result).toBe(freshToken);
expect(fetch).toHaveBeenCalledTimes(2); }); // — Test 4e
it('(e) logout() →
token cleared from memory, refresh endpoint called', async () => {
  const
token = makeJwt(3600);
  global.fetch =
vi.fn()
  .mockResolvedValueOnce({ ok: true, json: async () =>
({ accessToken: token }) } as Response)
  .mockResolvedValueOnce({ ok:
true } as Response); // logout endpoint
  await
service.login('user@havenworld.com', 'pass');
  await service.logout();
expect(service.getToken()).toBeNull();
expect(service.isAuthenticated()).toBe(false); // Confirm logout API was
called to clear httpOnly refresh cookie
expect(fetch).toHaveBeenLastCalledWith(
expect.stringContaining('/api/auth/logout'),
expect.objectContaining({ method: 'POST' })); }); // — Test 4f
it('(f)
isAuthenticated() → true when token present and not expired', async () => {
  const token = makeJwt(3600);
  global.fetch =
vi.fn().mockResolvedValueOnce({ ok: true, json: async () =>
({ accessToken: token }) } as Response);
  await
service.login('user@havenworld.com', 'pass');
expect(service.isAuthenticated()).toBe(true); }); // — Test 4g
it('(g) Token expiry:
isAuthenticated() false if within 60s of expiry', async () => { // Token
expires in 30 seconds – within the 60s safety window
  const
almostExpiredToken = makeJwt(30);
  global.fetch =
vi.fn().mockResolvedValueOnce({ ok: true, json: async () =>

```

```
({ accessToken: almostExpiredToken }) } as Response);    await
service.login('user@havenworld.com', 'pass');           // isAuthenticated should
treat "within 60s of expiry" as expired
expect(service.isAuthenticated()).toBe(false);    }); });
```

5. Unit Tests — SocketService MAC

SocketService wraps Socket.io's client and adds HavenWorld-specific behavior: authentication on connect, event queuing before connection is established, exponential backoff reconnection, and connection timeout. These tests mock `socket.io-client` entirely — no real network calls occur. This isolation is critical: a test that depends on a live Socket.io server is a fragile integration test, not a unit test.

File: `src/client/services/__tests__/socketService.test.ts`

```
// src/client/services/__tests__/socketService.test.ts import { describe, it,
expect, vi, beforeEach } from 'vitest'; import { SocketService,
NotAuthenticatedError } from '../socketService'; // Mock the entire
socket.io-client module vi.mock('socket.io-client', () => { const
mockSocket = { connect: vi.fn(), disconnect: vi.fn(), emit:
vi.fn(), on: vi.fn(), off: vi.fn(), connected: false, auth:
{} }; return { io: vi.fn(() => mockSocket), mockSocket }; }); import
{ io as mockIo, mockSocket } from 'socket.io-client';
describe('SocketService', () => { let service: SocketService;
beforeEach(() => { service = new SocketService(); vi.clearAllMocks();
(mockSocket as any).connected = false; }); // — Test 5a
it('(a) connect()
with valid token → socket.connect() called with auth', async () =>
{ await service.connect('valid.jwt.token');
expect(mockSocket.connect).toHaveBeenCalled(); // Token must be passed in
the auth handshake, not a query param
expect(mockSocket.auth).toMatchObject({ token: 'valid.jwt.token' }); });
// — Test 5b
it('(b)
connect() with no token → throws NotAuthenticatedError', async () =>
{ await
expect(service.connect('')).rejects.toThrow(NotAuthenticatedError);
expect(mockSocket.connect).not.toHaveBeenCalled(); }); // — Test 5c
it('(c) Reconnection:
on disconnect → exponential backoff attempted', async () =>
{ vi.useFakeTimers(); await
service.connect('valid.jwt.token'); // Simulate disconnect event from
socket.io const disconnectHandler = (mockSocket.on as Return<typeof
vi.fn>).mock.calls.find(([event]) => event === 'disconnect')?.[1];
disconnectHandler?.('transport error'); // First backoff: 1000ms
await vi.advanceTimersByTimeAsync(1000);
expect(mockSocket.connect).toHaveBeenCalledTimes(2); // Second backoff:
2000ms (exponential) disconnectHandler?.('transport error'); await
vi.advanceTimersByTimeAsync(2000);
expect(mockSocket.connect).toHaveBeenCalledTimes(3); vi.useRealTimers();
}); // — Test 5d
```

```

it('(d) emit() before connect → event queued, sent on connect', async () => {
  // Do NOT call connect() – socket is not yet established
  service.emit('PLAYER_MOVED', { x: 1, y: 0, z: 1 }); // Should NOT have
  emitted yet expect(mockSocket.emit).not.toHaveBeenCalled(); // Now
  connect – queued events should flush (mockSocket as any).connected =
  true; await service.connect('valid.jwt.token');
  expect(mockSocket.emit).toHaveBeenCalledWith('PLAYER_MOVED', { x: 1, y: 0, z:
  1 }); }); // — Test 5e

  it('(e) on() handler
  registration → handler called when event received', () => { const handler
  = vi.fn(); service.on('ROOM_JOINED', handler); // Simulate the
  server emitting ROOM_JOINED const registeredHandler = (mockSocket.on as
  ReturnType<typeof vi.fn>).mock.calls.find(([event]) => event ===
  'ROOM_JOINED')?.[1]; registeredHandler?.({ roomId: 'park', users: [] });
  expect(handler).toHaveBeenCalledWith({ roomId: 'park', users: [] }); });
  // — Test 5f

  it('(f)
  off() → handler deregistered, no longer called', () => { const handler =
  vi.fn(); service.on('CHAT_MESSAGE', handler);
  service.off('CHAT_MESSAGE', handler);
  expect(mockSocket.off).toHaveBeenCalledWith('CHAT_MESSAGE', handler); });
  // — Test 5g

  it('(g)
  Connection timeout (5s no WELCOME) → connection failed error', async () => {
  vi.useFakeTimers(); const connectPromise =
  service.connect('valid.jwt.token'); // Advance 5 seconds without a
  WELCOME event arriving await vi.advanceTimersByTimeAsync(5001);
  await expect(connectPromise).rejects.toThrow('Connection timeout');
  vi.useRealTimers(); }); });

```

6. Unit Tests — AvatarController MAC

AvatarController is the most complex client class to test because it directly manipulates Babylon.js scene objects. The solution is **NullEngine**: a Babylon.js engine that processes all logic but writes nothing to a GPU. You get a real, fully functional Babylon.js Scene with meshes, animations, and physics — just no pixels rendered. This is the correct approach for any Babylon.js logic test; do not mock Babylon.js itself.

File: src/client/game/__tests__/avatarController.test.ts

```

// src/client/game/__tests__/avatarController.test.ts import { describe, it,
expect, vi, beforeEach, afterEach, beforeAll, afterAll, beforeEach } from 'vitest'; import
{ NullEngine, Scene, AbstractMesh, AnimationGroup, MorphTarget } from
'@babylonjs/core'; import { AvatarController } from
'../avatarController'; // NullEngine: full Babylon.js logic, zero GPU/WebGL
dependency let engine: NullEngine; let scene: Scene; let controller:
AvatarController; beforeEach(() => { engine = new NullEngine(); scene =
new Scene(engine); }); afterEach(() => { scene.dispose();
engine.dispose(); }); beforeEach(() => { controller = new
AvatarController(scene); }); describe('AvatarController', () => { // —
Test 6a
  it('(a)
  Avatar loaded → mesh visible, idle animation playing', async () =>

```

```

{    await controller.loadAvatar('avatar_default');    const mesh =
scene.getMeshByName('Avatar_Root');    expect(mesh).not.toBeNull();
expect(mesh!.isEnabled()).toBe(true);    // Idle AnimationGroup should be
the only one currently playing    const playing =
scene.animationGroups.filter(g => g.isPlaying);    expect(playing.some(g =>
g.name === 'idle')).toBe(true);    });    // — Test 6b

    it('(b)
applyMorphTargets("slim") → morph target weight = 0.8', async () =>
{    await controller.loadAvatar('avatar_default');
controller.applyMorphTargets('slim');    const mesh =
scene.getMeshByName('Avatar_Body') as AbstractMesh;    const manager =
mesh?.morphTargetManager;    const slimTarget =
manager?.getTarget(    Array.from({ length: manager.numTargets }, (_, i)
=> i)    .find(i => manager?.getTarget(i).name === 'slim') ?? 0    );
expect(slimTarget?.influence).toBeCloseTo(0.8);    });    // — Test 6c

    it('(c)
applyMorphTargets("athletic") → athletic weight = 1.0, slim weight = 0',
async () => {    await controller.loadAvatar('avatar_default');
controller.applyMorphTargets('athletic');    const mesh =
scene.getMeshByName('Avatar_Body') as AbstractMesh;    const manager =
mesh?.morphTargetManager;    let athleticWeight = 0;    let slimWeight =
0;    for (let i = 0; i < (manager?.numTargets ?? 0); i++) {    const
target = manager!.getTarget(i);    if (target.name === 'athletic')
athleticWeight = target.influence;    if (target.name === 'slim')
slimWeight = target.influence;    }
expect(athleticWeight).toBeCloseTo(1.0);
expect(slimWeight).toBeCloseTo(0.0);    });    // — Test 6d

    it('(d)
playAnimation("walk") → walk group started, idle stopped', async () =>
{    await controller.loadAvatar('avatar_default');
controller.playAnimation('walk');    const walk =
scene.getAnimationGroupByName('walk');    const idle =
scene.getAnimationGroupByName('idle');
expect(walk?.isPlaying).toBe(true);    expect(idle?.isPlaying).toBe(false);
});    // — Test 6e

    it('(e) playAnimation("sit") → sit animation playing', async () =>
{    await controller.loadAvatar('avatar_default');
controller.playAnimation('sit');    const sit =
scene.getAnimationGroupByName('sit');    expect(sit?.isPlaying).toBe(true);
});    // — Test 6f

    it('(f) playAnimation("wave") → wave plays then returns to idle', async () =>
{    vi.useFakeTimers();    await controller.loadAvatar('avatar_default');
controller.playAnimation('wave');    const wave =
scene.getAnimationGroupByName('wave');
expect(wave?.isPlaying).toBe(true);    // Wave animation lasts ~2.5 seconds
— advance past it    await vi.advanceTimersByTimeAsync(3000);    const idle
= scene.getAnimationGroupByName('idle');
expect(idle?.isPlaying).toBe(true);    vi.useRealTimers();    });    // —
Test 6g

    it('(g)
Invalid animation name → stays on current, logs warning', async () =>
{    const warnSpy = vi.spyOn(console, 'warn').mockImplementation(() => {});
await controller.loadAvatar('avatar_default');
controller.playAnimation('idle');
controller.playAnimation('doesNotExist');    // Should still be on idle
(unchanged)    const idle = scene.getAnimationGroupByName('idle');
expect(idle?.isPlaying).toBe(true);
expect(warnSpy).toHaveBeenCalledWith(expect.stringContaining('doesNotExist'));
;    warnSpy.mockRestore();    });    // — Test 6h

    it('(h)
moveToPosition() → avatar moves via navmesh pathfinding', async () =>

```

```
{
  await controller.loadAvatar('avatar_default');
  vi.spyOn(controller as any, 'followNavmeshPath');
  controller.moveToPosition(5, 0, 5);
  expect(moveSpy).toHaveBeenCalled();
  expect(objectContaining({ x: expect.any(Number) })).toEqual(expect.objectContaining({ x: expect.any(Number) }));
});
```

7. Unit Tests — RoomLoader MAC

RoomLoader is responsible for fetching GLB files, adding their meshes to the Babylon.js scene, extracting the navigation mesh (by name convention `_navmesh`), and cleanly unloading rooms without memory leaks. The memory leak test is particularly important: a room that does not fully dispose its meshes will cause the GPU heap to grow over a long session, eventually crashing the browser tab.

File: `src/client/game/__tests__/roomLoader.test.ts`

```
// src/client/game/__tests__/roomLoader.test.ts
import { describe, it, expect, vi, beforeEach, afterEach } from 'vitest';
import { NullEngine, Scene } from '@babylonjs/core';
import { RoomLoader } from '../roomLoader';

let engine: NullEngine;
let scene: Scene;
let loader: RoomLoader;

beforeEach(() => {
  engine = new NullEngine();
  scene = new Scene(engine);
  loader = new RoomLoader(scene);
});

afterEach(() => {
  scene.dispose();
  engine.dispose();
});

describe('RoomLoader', () => {
  // — Test 7a
  it('a) loadRoom("park") → root mesh added to scene', async () => {
    // Mock SceneLoader.ImportMeshAsync (network call to GLB file)
    vi.mock('@babylonjs/core/Loading/sceneLoader', () => ({
      SceneLoader: {
        ImportMeshAsync: vi.fn().mockResolvedValue({
          meshes: [{ name: 'Park_Root', isEnabled: () => true }],
          animationGroups: [],
        }),
      },
    }));

    await loader.loadRoom('park');
    const root = scene.getMeshByName('Park_Root');
    expect(root).not.toBeNull();
  });

  // — Test 7b
  it('b) extractNavmesh() → finds mesh named "_navmesh"', async () => {
    await loader.loadRoom('park');
    const navmesh = loader.extractNavmesh();
    expect(navmesh).not.toBeNull();
    expect(navmesh!.name).toBe('_navmesh');
  });

  // — Test 7c
  it('c) loadRoom with missing GLB → graceful fallback to default room', async () => {
    const warnSpy = vi.spyOn(console, 'warn').mockImplementation(() => {});
    // Simulate a 404 on the GLB fetch
    vi.mocked(SceneLoader.ImportMeshAsync).mockRejectedValueOnce(new Error('404 Not Found'));
    await loader.loadRoom('nonexistentRoom');
    // Should fall back to 'default' room, not throw
    expect(warnSpy).toHaveBeenCalled();
    warnSpy.mockRestore();
  });

  // — Test 7d
  it('d) Room metadata loaded: spawn points and furniture anchors present', async () => {
    const metadata = await loader.loadRoomMetadata('park');
    expect(metadata.spawnPoints.length).toBeGreaterThan(0);
    expect(metadata.furnitureAnchors).toBeDefined();
    expect(Array.isArray(metadata.furnitureAnchors)).toBe(true);
  });

  // — Test 7e
  it('e) Room'

```

```

unload → all meshes disposed, no memory leak', async () => {    await
loader.loadRoom('park');    const meshCountBefore = scene.meshes.length;
await loader.unloadRoom();    // After unload, scene should have FEWER
meshes than before room was loaded    // (equal if room was already empty,
but never more)
expect(scene.meshes.length).toBeLessThanOrEqual(meshCountBefore);    });    //
— Test 7f —————    it('(f)
Switch room → old meshes disposed before new room loaded', async () =>
{    const disposeSpy = vi.fn();    vi.spyOn(loader as any,
'disposeCurrentRoom').mockImplementation(disposeSpy);    await
loader.loadRoom('park');    await loader.loadRoom('beachhouse');    //
switching rooms    // Dispose must have been called before the new room
loads    expect(disposeSpy).toHaveBeenCalledOnce();    }); });

```

8. Unit Tests — FurnitureManager

MAC

FurnitureManager handles the personal room decorating feature. Tests cover placement, overlap detection, removal, the 50-item limit (a game design constraint that must be enforced client-side before sending to the server), and round-trip serialization/restoration of saved room layouts. Serialization is critical: if `serialize()` or `restore()` is lossy, players lose their room decorations on reload.

File: `src/client/game/__tests__/furnitureManager.test.ts`

```

// src/client/game/__tests__/furnitureManager.test.ts import { describe, it,
expect, vi, beforeAll, afterAll, beforeEach } from 'vitest'; import
{ NullEngine, Scene, Vector3 } from '@babylonjs/core'; import
{ FurnitureManager, FurnitureOverlapError, FurnitureLimitError } from
'../furnitureManager'; let engine: NullEngine; let scene: Scene; let
manager: FurnitureManager; beforeAll(() => {    engine = new NullEngine();
scene = new Scene(engine); }); afterAll(() => { scene.dispose();
engine.dispose(); }); beforeEach(() => { manager = new
FurnitureManager(scene); }); describe('FurnitureManager', () => {    // —
Test 8a —————    it('(a)
placeFurniture() → mesh at correct world position', async () => {    const
pos = new Vector3(2, 0, 3);    const instance = await
manager.placeFurniture('chair_oak', pos);    const mesh =
scene.getMeshByUniqueId(instance.meshUniqueId);
expect(mesh?.position.x).toBeCloseTo(2);
expect(mesh?.position.z).toBeCloseTo(3);    });    // — Test 8b
    it('(b)
placeFurniture at occupied position → throws FurnitureOverlapError', async ()
=> {    const pos = new Vector3(0, 0, 0);    await
manager.placeFurniture('table_round', pos);    // Same position – should be
rejected by overlap detection    await
expect(
    manager.placeFurniture('chair_oak',
pos)    ).rejects.toThrow(FurnitureOverlapError);    });    // — Test 8c
    it('(c)
removeFurniture() → mesh disposed from scene', async () => {    const

```

```

instance = await manager.placeFurniture('lamp_floor', new Vector3(5, 0, 5));
const meshId = instance.meshUniqueId;
manager.removeFurniture(instance.instanceId);
expect(scene.getMeshByUniqueId(meshId)).toBeNull(); }); // — Test 8d
it('(d)
moveFurniture() → mesh position updated', async () => { const instance =
await manager.placeFurniture('rug_small', new Vector3(1, 0, 1));
manager.moveFurniture(instance.instanceId, new Vector3(8, 0, 8)); const
mesh = scene.getMeshByUniqueId(instance.meshUniqueId);
expect(mesh?.position.x).toBeCloseTo(8);
expect(mesh?.position.z).toBeCloseTo(8); }); // — Test 8e
it('(e) 51st item
rejected – personal room limit is 50', async () => { // Place 50 items at
non-overlapping positions for (let i = 0; i < 50; i++) { await
manager.placeFurniture('cushion', new Vector3(i * 2, 0, 0)); } //
51st should be rejected await
expect( manager.placeFurniture('cushion', new Vector3(200, 0,
0)) ).rejects.toThrow(FurnitureLimitError); }); // — Test 8f
it('(f) serialize() →
returns JSON array of all placed items with positions', async () => {
{ await manager.placeFurniture('bookshelf', new Vector3(0, 0, 0));
await manager.placeFurniture('plant_fern', new Vector3(3, 0, 0)); const
saved = manager.serialize(); expect(saved).toHaveLength(2);
expect(saved[0]).toMatchObject({ itemId: 'bookshelf', position: {
x: 0, y: 0, z: 0 }, }); }); // — Test 8g
it('(g)
restore(savedLayout) → reconstructs scene from saved JSON', async () => {
const layout = [ { itemId: 'sofa_3seat', instanceId: 'inst_001',
position: { x: 1, y: 0, z: 2 }, rotation: 0 }, { itemId:
'coffee_table', instanceId: 'inst_002', position: { x: 1, y: 0, z: 4 },
rotation: 0 }, ]; await manager.restore(layout);
expect(scene.getMeshByName('sofa_3seat_inst_001')).not.toBeNull();
expect(scene.getMeshByName('coffee_table_inst_002')).not.toBeNull(); }); }
;

```

9. Unit Tests — AudioEngine MAC

HavenWorld's AudioEngine synthesizes all sounds using the Web Audio API's oscillator graph — zero audio file downloads, zero CDN dependency, zero latency on first play. jsdom does not implement Web Audio, so we mock AudioContext entirely with `vi.fn()`. The goal is not to test that oscillators produce correct waveforms (that is the browser's job), but that AudioEngine calls the Web Audio API with the correct frequency parameters, gain values, and envelope shapes for each named sound.

File: `src/client/game/__tests__/audioEngine.test.ts`

```

// src/client/game/__tests__/audioEngine.test.ts import { describe, it,
expect, vi, beforeEach } from 'vitest'; import { AudioEngine } from
'../audioEngine'; // Mock Web Audio API nodes const mockGainNode =
{
  connect: vi.fn(),
  gain: { setValueAtTime: vi.fn(),
exponentialRampToValueAtTime: vi.fn(), value: 1 },
}; const mockOscillator =

```

```

{  connect: vi.fn(),  start: vi.fn(),  stop: vi.fn(),  type: 'sine',
frequency: { setValueAtTime: vi.fn(), exponentialRampToValueAtTime: vi.fn(),
value: 440 },  };  const mockContext = {  createOscillator: vi.fn(() =>
({ ...mockOscillator })),  createGain: vi.fn(() => ({ ...mockGainNode })),
destination: {},  currentTime: 0,  state: 'running',  };  // Replace
browser's AudioContext with our mock global.AudioContext = vi.fn(() =>
mockContext) as any;  describe('AudioEngine', () => {  let audio:
AudioEngine;  beforeEach(() => {  audio = new AudioEngine();
vi.clearAllMocks();
mockContext.createOscillator.mockReturnValue({ ...mockOscillator });
mockContext.createGain.mockReturnValue({ ...mockGainNode });  });  // —
Test 9a  _____  it('(a)
playSound("footstep") → OscillatorNode created', () =>
{  audio.playSound('footstep');
expect(mockContext.createOscillator).toHaveBeenCalled();  });  // — Test
9b  _____  it('(b)
playSound("coin_pickup") → ascending E5→B5 tone (329.6Hz → 493.9Hz)', () => {
audio.playSound('coin_pickup');  const osc =
mockContext.createOscillator.mock.results[0].value;  // Starting frequency
should be E5 (329.6 Hz)
expect(osc.frequency.setValueAtTime).toHaveBeenCalledWith(  expect.close
To(329.6, 0),  expect.any(Number)  );  // Ramp up to B5 (493.9 Hz)
expect(osc.frequency.exponentialRampToValueAtTime).toHaveBeenCalledWith(
expect.closeTo(493.9, 0),  expect.any(Number)  );  });  // — Test
9c  _____  it('(c)
playSound("chat_message") → crystal chime synthesized', () =>
{  audio.playSound('chat_message');  // Chat chime uses a sine wave
oscillator at a high frequency  const osc =
mockContext.createOscillator.mock.results[0].value;
expect(osc.type).toBe('sine');
expect(osc.frequency.setValueAtTime).toHaveBeenCalledWith(  expect.toBeG
reaterThan(800),  expect.any(Number)  );  });  // — Test 9d
_____  it('(d) mute() →
master gain set to 0', () => {  audio.mute();  const gain =
mockContext.createGain.mock.results[0]?.value ?? mockGainNode;
expect(gain.gain.value).toBe(0);
expect(audio.isMuted()).toBe(true);  });  // — Test 9e
_____  it('(e) unmute() →
gain restored to previous level', () => {  audio.setVolume(0.7);
audio.mute();  audio.unmute();  expect(audio.isMuted()).toBe(false);
// Volume should be back to 0.7, not 1.0 (don't reset to default)
expect(audio.getVolume()).toBeCloseTo(0.7);  });  // — Test 9f
_____  it('(f) isMuted() →
correct boolean state', () => {  expect(audio.isMuted()).toBe(false);
audio.mute();  expect(audio.isMuted()).toBe(true);  audio.unmute();
expect(audio.isMuted()).toBe(false);  });  // — Test 9g
_____  it('(g) playSound
when muted → AudioContext still called, gain is 0', () => {  audio.mute();
audio.playSound('footstep');  // We do NOT skip sound creation when muted
— skipping causes audible "pop"  // on unmute if sounds were supposed to
loop. Just set gain to 0 instead.
expect(mockContext.createOscillator).toHaveBeenCalled();  });  // — Test
9h  _____  it('(h) Unknown
sound name → no-op, no error thrown', () => {  expect(() =>
audio.playSound('unknown_sfx_xyz')).not.toThrow();
expect(mockContext.createOscillator).not.toHaveBeenCalled();  });  });

```

10. Unit Tests — ChatOverlay MAC

ChatOverlay is a pure HTML/CSS/TS UI component — no Babylon.js involved. We test it with `@testing-library/dom` and `@testing-library/user-event` against the jsdom environment. The XSS prevention test (links rendered as text) is non-negotiable: chat is the highest-risk injection surface in a social game. Never trust message content from the server.

File: `src/client/ui/__tests__/chatOverlay.test.ts`

```
// src/client/ui/__tests__/chatOverlay.test.ts import { describe, it, expect,
vi, beforeEach } from 'vitest'; import { getByRole, getByTestId, fireEvent,
waitFor } from '@testing-library/dom'; import userEvent from '@testing-
library/user-event'; import { ChatOverlay } from '../chatOverlay';
describe('ChatOverlay', () => { let container: HTMLElement; let overlay:
ChatOverlay; let onSubmit: ReturnType<typeof vi.fn>; beforeEach(() => {
container = document.createElement('div');
document.body.appendChild(container); onSubmit = vi.fn(); overlay =
new ChatOverlay(container, { onSubmit }); overlay.mount(); }); // —
Test 10a _____ it('(a)
Render → input field and message list present in DOM', () => { const
input = container.querySelector('[aria-label="Chat message"]'); const
list = getByTestId(container, 'chat-messages');
expect(input).not.toBeNull(); expect(list).not.toBeNull(); }); // —
Test 10b _____ it('(b) Type
message + Enter → onSubmit called with trimmed text', async () => { const
user = userEvent.setup(); const input = container.querySelector('[aria-
label="Chat message"]') as HTMLInputElement; await user.click(input);
await user.type(input, ' Hello world! '); await
user.keyboard('{Enter}'); expect(onSubmit).toHaveBeenCalledWith('Hello
world!'); }); // — Test 10c
_____ it('(c) Empty message
+ Enter → onSubmit NOT called', async () => { const user =
userEvent.setup(); const input = container.querySelector('[aria-
label="Chat message"]') as HTMLInputElement; await user.click(input);
await user.type(input, ' '); // only whitespace await
user.keyboard('{Enter}'); expect(onSubmit).not.toHaveBeenCalled(); });
// — Test 10d _____ it('(d)
Message exceeding 200 chars → submit button disabled', async () =>
{ const user = userEvent.setup(); const input =
container.querySelector('[aria-label="Chat message"]') as HTMLInputElement;
const submit = container.querySelector('[data-testid=chat-submit]') as
HTMLButtonElement; const longMsg = 'a'.repeat(201); await
user.type(input, longMsg);
expect(submit.disabled).toBe(true); }); // — Test 10e
_____ it('(e) Receive
message → appears in list with username and timestamp', () =>
{ overlay.receiveMessage({ username: 'Aria', text: 'Hey
everyone!', timestamp: new Date('2026-09-18T16:39:00Z'), });
const list = getByTestId(container, 'chat-messages');
expect(list.textContent).toContain('Aria');
expect(list.textContent).toContain('Hey everyone!');
expect(list.textContent).toMatch(/\d{1,2}:\d{2}/); // time
format }); // — Test 10f
_____ it('(f) 50 messages →
```

```

oldest removed (max 50 visible)', () => {      for (let i = 1; i <= 51; i++) {
overlay.receiveMessage({ username: 'Bot', text: `Message ${i}`, timestamp:
new Date() });      }      const list = getByTestId(container, 'chat-
messages');      const items = list.querySelectorAll('[data-testid=chat-
message]');      expect(items.length).toBe(50);      // Message 1 should be
gone; Message 2 is now oldest visible
expect(list.textContent).not.toContain('Message 1');
expect(list.textContent).toContain('Message 2');
expect(list.textContent).toContain('Message 51');  });  // — Test 10g
it('(g) Rate limit
active → input shows cooldown timer display', async () =>
{      vi.useFakeTimers();      overlay.activateRateLimit(5); // 5-second
cooldown      const cooldown = container.querySelector('[data-testid=chat-
cooldown]');      expect(cooldown).not.toBeNull();
expect(cooldown?.textContent).toContain('5');      await
vi.advanceTimersByTimeAsync(1000);
expect(cooldown?.textContent).toContain('4');      vi.useRealTimers();  });
// — Test 10h  it('(h)
Link in message → rendered as TEXT, NOT as anchor (XSS prevention)', () => {
overlay.receiveMessage({      username: 'Attacker',      text: 'Click here:
https://evil.example.com/steal-cookies',      timestamp: new Date(),  });
const list = getByTestId(container, 'chat-messages');      // No <a> tags
should exist in the chat — URLs stay as plaintext      const anchors =
list.querySelectorAll('a');      expect(anchors.length).toBe(0);
expect(list.textContent).toContain('https://evil.example.com/steal-cookies');
}); });

```

11. Babylon.js Scene Tests — NullEngine MAC

11.1 What Is NullEngine?

Babylon.js's `NullEngine` is an official, first-party testing engine that implements the full Babylon.js rendering API without creating a WebGL context. It processes every frame — scene graph traversal, animation updates, physics simulation, event dispatching — but discards the draw calls instead of sending them to a GPU. This means you can test Babylon.js scene logic in any Node.js-compatible environment (Vitest, Jest, CI server) without a display or GPU. `NullEngine` is bundled with `@babylonjs/core` at no extra cost.

⚠ **Warning: Do Not Mock Babylon.js Itself**

A common mistake is to mock `@babylonjs/core` classes with `vi.mock()`. This defeats the purpose: you end up testing your own mock, not your scene logic. Always use a real `NullEngine + Scene`. Only mock the *inputs* to your scene logic (GLB loaders, network calls, timers).

File: `__tests__/scene/roomScene.test.ts`

```
// __tests__/scene/roomScene.test.ts import { describe, it, expect, vi,
beforeAll, afterAll, beforeEach } from 'vitest'; import { NullEngine,
Scene, Color3, ArcRotateCamera, HemisphericLight, PhysicsImpostor, Vector3,
Color4, MeshBuilder, } from '@babylonjs/core'; import { RoomScene } from
'../../src/client/game/roomScene'; let engine: NullEngine; let scene: Scene;
let roomScene: RoomScene; beforeAll(() => { engine = new NullEngine(); });
afterAll(() => { engine.dispose(); }); beforeEach(() => { scene = new
Scene(engine); roomScene = new RoomScene(scene); }); describe('RoomScene -
NullEngine Tests', () => { // — Test 11a
it('(a) Scene creation
→ 1 camera, ambient light set', () => { roomScene.initialize();
expect(scene.cameras.length).toBe(1);
expect(scene.cameras[0]).toBeInstanceOf(ArcRotateCamera);
expect(scene.lights.length).toBeGreaterThan(0);
expect(scene.lights[0]).toBeInstanceOf(HemisphericLight); }); // — Test
11b
it('(b)
loadAvatarMesh() → mesh in scene, morph targets available', async () => {
roomScene.initialize(); await roomScene.loadAvatarMesh('player_123');
const avatarMesh = scene.getMeshByName('Avatar_Root');
expect(avatarMesh).not.toBeNull();
expect(avatarMesh?.morphTargetManager).not.toBeNull(); }); // — Test
11c
it('(c) Avatar
position update → Vector3 updated correctly', async () =>
{ roomScene.initialize(); await
roomScene.loadAvatarMesh('player_123');
roomScene.setAvatarPosition('player_123', new Vector3(5, 0, 10)); const
mesh = scene.getMeshByName('Avatar_Root');
expect(mesh?.position.x).toBeCloseTo(5);
expect(mesh?.position.z).toBeCloseTo(10); }); // — Test 11d
it('(d) Lighting
presets: Day ambient = bright white, Night = dark blue', () =>
{ roomScene.initialize(); roomScene.applyMoodPreset('day');
const dayLight = scene.lights[0] as HemisphericLight;
expect(dayLight.diffuse.r).toBeGreaterThan(0.8); // bright
roomScene.applyMoodPreset('night'); const nightLight = scene.lights[0] as
HemisphericLight;
expect(nightLight.diffuse.b).toBeGreaterThan(nightLight.diffuse.r); // blue
bias }); // — Test 11e
it('(e) Night preset →
volumetric fog enabled (FOGMODE_EXP2)', () => { roomScene.initialize();
roomScene.applyMoodPreset('night'); // EXP2 fog for atmospheric depth on
the Night/Cyber presets expect(scene.fogMode).toBe(Scene.FOGMODE_EXP2);
expect(scene.fogColor).toMatchObject(expect.objectContaining({ r:
expect.any(Number), g: expect.any(Number), b:
expect.any(Number) })); }); // — Test 11f
it('(f) Physics
```

```

impostor on ground mesh → PhysicsImpostor attached', () =>
{
  roomScene.initialize();      const ground = roomScene.getGroundMesh();
  expect(ground?.physicsImpostor).toBeInstanceOf(PhysicsImpostor);
  expect(ground?.physicsImpostor?.type).toBe(PhysicsImpostor.BoxImpostor);  })
;    // — Test 11g —————
it('(g) Pet FSM: idle → wander transition after 5 virtual seconds', async ()
=> {
  vi.useFakeTimers();      roomScene.initialize();      const pet =
roomScene.spawnPet('corgi_01', new Vector3(0, 0, 0));
  expect(pet.state).toBe('idle');      // Advance 5 virtual seconds (pet idle
timeout)      await vi.advanceTimersByTimeAsync(5000);
  expect(pet.state).toBe('wander');      vi.useRealTimers();  });    // — Test
11h —————      it('(h) Pet follow
owner → pet mesh moves towards owner each frame', () =>
{
  roomScene.initialize();      const ownerPos = new Vector3(10, 0, 10);
  const pet = roomScene.spawnPet('corgi_01', new Vector3(0, 0, 0));
  pet.setState('follow');      roomScene.setOwnerPosition(ownerPos);      //
Simulate 10 render frames      for (let i = 0; i < 10; i++)
  {
    scene.render();      }      // Pet should have moved closer to the
owner after 10 frames      const petPos =
scene.getMeshByName(`pet_corgi_01`)?.position;
  expect(petPos?.x).toBeGreaterThan(0);
  expect(petPos?.z).toBeGreaterThan(0);  });    // — Test 11i
—————      it('(i) Navmesh
extraction → walkable area polygon count > 0', async () =>
{
  roomScene.initialize();      await roomScene.loadRoom('park');
  const navmesh = roomScene.extractNavmesh();
  expect(navmesh).not.toBeNull();      // Navmesh must have actual geometry – 0
polygons means pathfinding is broken
  expect(navmesh!.getTotalVertices()).toBeGreaterThan(0);  }); });

```

12. Playwright E2E — Full Game Flow MAC

This is the most valuable E2E test: it walks through the complete new-user journey from registration to logout. Every step maps to a real user action. If any step fails in CI, a bug has broken a critical user path. The test uses the Page Object Model classes defined in Section 3.

File: e2e/fullFlow.spec.ts

```

// e2e/fullFlow.spec.ts import { test, expect, Page } from
'@playwright/test'; import { LoginPage } from './pages/LoginPage'; import
{ LobbyPage } from './pages/LobbyPage'; import { RoomPage } from
'./pages/RoomPage'; const TEST_EMAIL = `testflow_${
Date.now()}@havenworld.test`; const TEST_PASSWORD = 'TestPass!2026';
test.describe('Full Game Flow – Registration to Logout', () =>
{
  test('Step 1 – Navigate to app root, redirects to login', async
({ page }) => {
    await page.goto('/');      await
expect(page).toHaveURL(/\/login/);  });    test('Step 2 – Registration →
verify form and submit', async ({ page }) => {
    const loginPage = new

```

```

LoginPage(page);      await loginPage.goto();      await
loginPage.registerLink.click();      await page.waitForURL('**/register');
await page.fill('[name=email]', TEST_EMAIL);      await
page.fill('[name=password]', TEST_PASSWORD);      await
page.fill('[name=confirmPassword]', TEST_PASSWORD);      // Use
TEST_INVITE_CODE from .env.test – invite system is required in HavenWorld
alpha      await page.fill('[name=inviteCode]',
process.env.TEST_INVITE_CODE!);      await page.click('button[type=submit]');
// CI: Call API directly to skip email verification step      await fetch(`${
{process.env.TEST_API_URL}/api/test/verify-email`, {      method: 'POST',
headers: { 'Content-Type': 'application/json' },      body: JSON.stringify({
email: TEST_EMAIL })),      });      });      test('Step 3 – Login → lobby loads
with avatar visible', async ({ page }) => {      const loginPage = new
LoginPage(page);      const lobbyPage = new LobbyPage(page);      await
loginPage.goto();      await loginPage.login(TEST_EMAIL,
TEST_PASSWORD);      // Should redirect to lobby after successful login
await expect(page).toHaveURL(/\/lobby/, { timeout: 10_000 });      await
lobbyPage.waitForLoad();      await
expect(lobbyPage.avatarCanvas.toBeVisible());      });      test('Step 4 – Join
Haven Park → room canvas loads', async ({ page }) => {      // Use saved auth
state (from globalSetup) to skip re-logging in      const lobbyPage = new
LobbyPage(page);      const roomPage = new RoomPage(page);      await
page.goto('/lobby');      await lobbyPage.waitForLoad();      await
lobbyPage.havenParkButton.click();      await roomPage.waitForRoomLoad();
await expect(roomPage.gameCanvas.toBeVisible());      });      test('Step 5 –
Click canvas → avatar moves, PLAYER_MOVED event emitted', async ({ page }) =>
{      const roomPage = new RoomPage(page);      await page.goto('/lobby');
await page.locator('[data-testid=room-haven-park]').click();      await
roomPage.waitForRoomLoad();      // Listen for the PLAYER_MOVED socket event
via page.evaluate      const socketEventPromise = page.evaluate(() =>
{      return new Promise<{x: number, z: number}>(resolve =>
{      (window as any).__havenSocket.once('PLAYER_MOVED', (data: any) =>
resolve(data));      });      });      // Click roughly center-left of canvas
to trigger pathfinding move      await roomPage.clickCanvas(200, 300);
const movedData = await socketEventPromise;      expect(typeof
movedData.x).toBe('number');      expect(typeof movedData.z).toBe('number');
});      test('Step 6 – Send chat message → appears in overlay', async
({ page }) => {      const roomPage = new RoomPage(page);      await
page.goto('/lobby');      await page.locator('[data-testid=room-haven-
park]').click();      await roomPage.waitForRoomLoad();      await
roomPage.sendChatMessage('Hello from Playwright!');      await
expect(roomPage.chatMessages).toContainText('Hello from Playwright!', {
timeout: 5_000,      });      });      test('Step 7 – Logout → redirected to login,
storage cleared', async ({ page }) => {      await page.goto('/lobby');
await page.locator('[data-testid=logout-btn]').click();      await
expect(page).toHaveURL(/\/login/);      // Verify no tokens leaked into
localStorage or sessionStorage      const storage = await page.evaluate(() =>
({      local: { ...localStorage },      session: { ...sessionStorage },
}));      expect(Object.keys(storage.local)).not.toContain('accessToken');
expect(Object.keys(storage.session)).not.toContain('accessToken');      }); });

```

13. Playwright E2E — Two-Player Interaction MAC

Multiplayer synchronization is HavenWorld's core value proposition. This test launches two completely independent browser contexts in the same test, simulates two users entering the same room, and verifies that each user's actions are visible to the other. Use `browser.newContext()` — not `page.newPage()` — so each session has completely isolated cookies, `localStorage`, and authentication state.

File: `e2e/multiplayer.spec.ts`

```
// e2e/multiplayer.spec.ts import { test, expect, Browser, Page } from
'@playwright/test'; import { RoomPage } from './pages/RoomPage';
test.describe('Two-Player Interaction', () => { let browser: Browser; let
pageA: Page; let pageB: Page; test.beforeAll(async ({ browser: b }) => {
browser = b; // Create two isolated contexts – separate cookies,
localStorage, sessions const contextA = await
browser.newContext({ storageState: 'e2e/.auth/user_a.json', });
const contextB = await browser.newContext({ storageState:
'e2e/.auth/user_b.json', }); pageA = await contextA.newPage();
pageB = await contextB.newPage(); }); test.afterAll(async () =>
{ await pageA.close(); await pageB.close(); }); test('Both users
join Haven Park', async () => { const roomA = new RoomPage(pageA);
const roomB = new RoomPage(pageB); // Navigate both users to Haven Park
simultaneously await Promise.all([ pageA.goto('/lobby').then(() =>
pageA.locator('[data-testid=room-haven-park]').click()),
pageB.goto('/lobby').then(() => pageB.locator('[data-testid=room-haven-
park]').click()), ]); await Promise.all([roomA.waitForRoomLoad(),
roomB.waitForRoomLoad()]); await expect(roomA.gameCanvas).toBeVisible();
await expect(roomB.gameCanvas).toBeVisible(); }); test('User A moves →
User B sees avatar at new position', async () => { const roomA = new
RoomPage(pageA); // User A clicks to move await
roomA.clickCanvas(300, 300); // User B's page should receive
PLAYER_MOVED and update avatar position await pageB.waitForFunction(
() => (window as any).__havenOtherAvatars?.['testuser_a']?.moved === true,
{ timeout: 8_000 } ); const avatarPos = await
pageB.evaluate(() => (window as any).__havenOtherAvatars?.
['testuser_a']?.position ); expect(typeof
avatarPos?.x).toBe('number'); }); test('User B sends chat → User A sees
the message', async () => { const roomB = new RoomPage(pageB); const
roomA = new RoomPage(pageA); await roomB.sendMessage('Hey User A,
can you see this?'); await expect(roomA.chatMessages).toContainText('Hey
User A, can you see this?', { timeout: 8_000, }); });
test('User A initiates trade with User B → trade modal appears on both',
async () => { // User A sends trade request via game UI await
pageA.locator('[data-testid=trade-btn-testuser_b]').click(); await
pageA.locator('[data-testid=trade-confirm]').click(); // Verify trade
modal opens on BOTH screens await expect(pageA.locator('[data-
testid=trade-modal]')).toBeVisible({ timeout: 5_000 }); await
expect(pageB.locator('[data-testid=trade-modal]')).toBeVisible({ timeout:
5_000 }); }); test('Both lock + confirm trade → items swapped in UI',
async () => { // Both users lock their offers await Promise.all([
pageA.locator('[data-testid=trade-lock]').click(),
```

```

pageB.locator('[data-testid=trade-lock]').click(),    });    // Both
confirm    await Promise.all([    pageA.locator('[data-testid=trade-
final-confirm]').click(),    pageB.locator('[data-testid=trade-final-
confirm]').click(),    });    // Verify trade result via inventory API –
server is source of truth    const inventoryB = await fetch(`$
{process.env.TEST_API_URL}/api/test/inventory/testuser_b`    ).then(r =>
r.json());
expect(inventoryB.items).toContainEqual(    expect.objectContaining({ trad
edFrom: 'testuser_a' })    );    }); });

```

14. Playwright E2E — Avatar Customization MAC

This test validates that avatar customization persists across a hard reload. A hard reload clears all JavaScript state, forcing the app to re-fetch from the server. If the server did not save the customization, the avatar will revert — a high-visibility bug in a social game. The test must use `page.reload({ waitUntil: 'networkidle' })` rather than navigation, to simulate exactly what a user experiences when their browser crashes and reopens the tab.

File: `e2e/avatarCustomizer.spec.ts`

```

// e2e/avatarCustomizer.spec.ts import { test, expect } from
'@playwright/test'; test.use({ storageState: 'e2e/.auth/user_a.json' });
test.describe('Avatar Customization Persistence', () => {    test('Customize
avatar → save → reload → verify persisted', async ({ page }) => {    // —
Step 1: Open avatar customizer    await
page.goto('/avatar-customizer');    await expect(page.locator('[data-
testid=avatar-customizer]')).toBeVisible();    // — Step 2: Move body type
slider to "Athletic"    const bodyTypeSlider =
page.locator('[data-testid=body-type-slider]');    await
bodyTypeSlider.fill('1'); // 0=slim, 0.5=default, 1=athletic    await
expect(bodyTypeSlider).toHaveValue('1');    // — Step 3: Change hair color
to blue    const hairColorPicker =
page.locator('[data-testid=hair-color-picker]');    await
hairColorPicker.fill('#0000ff');    await
expect(hairColorPicker).toHaveValue('#0000ff');    // — Step 4: Save the
customization    await page.locator('[data-
testid=save-avatar-btn]').click();    // Wait for save confirmation toast
await expect(page.locator('[data-testid=save-success-toast]')).toBeVisible({
timeout: 5_000,    });    // — Step 5: Hard reload (clears all JS heap
state)    await page.reload({ waitUntil: 'networkidle' });
// — Step 6: Verify avatar customizer loads saved state    await expect(page.locator('[data-testid=avatar-customizer]')).toBeVisible({
timeout: 8_000,    });    // Body type should still be Athletic (value: 1)
const bodyTypeValue = await page.locator('[data-testid=body-type-
slider]').inputValue();
expect(parseFloat(bodyTypeValue)).toBeCloseTo(1.0, 1);    // Hair color
should still be blue    const hairColor = await page.locator('[data-

```

```

testid=hair-color-picker']).inputValue();
expect(hairColor.toLowerCase()).toBe('#0000ff'); }); test('Avatar
customizer preview updates live (no save required)', async ({ page }) => {
await page.goto('/avatar-customizer'); // Changing a value should
immediately update the 3D preview canvas const previewCanvas =
page.locator('[data-testid=avatar-preview-canvas]'); const initialHash =
await previewCanvas.evaluate( (el: HTMLCanvasElement) =>
el.toDataURL().slice(0, 100) ); await page.locator('[data-
testid=body-type-slider]').fill('0'); // change to slim // Wait for
canvas to re-render (Babylon.js frame) await page.waitForTimeout(500);
const updatedHash = await previewCanvas.evaluate( (el:
HTMLCanvasElement) => el.toDataURL().slice(0, 100) ); // Canvas
content should have changed (morph target applied)
expect(updatedHash).not.toBe(initialHash); }); });

```

15. Visual Regression Testing MAC

Visual regression tests catch unintended UI changes — a CSS refactor that misaligns the shop modal, a font-size change that clips text in the chat overlay, or a Babylon.js upgrade that changes default lighting. Playwright's built-in `toHaveScreenshot()` stores baseline PNG snapshots and compares them pixel-by-pixel on every subsequent run.

⚠ Important: Canvas vs. Overlay Testing Strategy

The Babylon.js canvas may produce minor GPU-driver-dependent pixel differences between machines (anti-aliasing, sub-pixel rendering). Always test the HTML overlay UI elements (chat, shop modal, login form) as screenshots — they are deterministic. For the game canvas, either use a generous pixel diff threshold (`maxDiffPixelRatio: 0.05`) or mask the canvas entirely and test canvas contents separately via Babylon.js scene unit tests (Section 11).

File: `e2e/visual/rooms.spec.ts`

```

// e2e/visual/rooms.spec.ts import { test, expect } from '@playwright/test';
// Only run visual tests in Chromium – cross-browser pixel diffs are too
noisy test.use({ browserName: 'chromium' }); test.use({ storageState:
'e2e/.auth/user_a.json' }); test.describe('Visual Regression – Room UI', ()
=> { test('Haven Park chat overlay – baseline screenshot', async
({ page }) => { await page.goto('/lobby'); await page.locator('[data-
testid=room-haven-park]').click(); // Wait for room to fully stabilize
before screenshotting await page.waitForFunction(() => (window as

```

```

any().__havenRoomReady === true);      await page.waitForTimeout(1000); //
extra settle time for animations        // Mask the Babylon.js canvas – GPU
pixel differences make it unreliable    await expect(page.locator('[data-
testid=chat-overlay]')).toHaveScreenshot('haven-park-chat-
overlay.png', { maxDiffPixelRatio: 0.01 } // 1% pixel tolerance );
}); test('Avatar customizer UI – default avatar screenshot', async ({ page
}) => {      await page.goto('/avatar-customizer');      await
expect(page.locator('[data-testid=avatar-customizer]')).toBeVisible();
await page.waitForTimeout(800);      await expect(page.locator('[data-
testid=customizer-panel]')).toHaveScreenshot('avatar-customizer-
default.png', { maxDiffPixelRatio: 0.02 } ); }); test('Shop
modal – item grid renders correctly', async ({ page }) => {      await
page.goto('/lobby');      await page.locator('[data-testid=open-shop-
btn]').click();      await expect(page.locator('[data-testid=shop-
modal]')).toBeVisible();      await expect(page.locator('[data-testid=shop-
modal]')).toHaveScreenshot('shop-modal-item-grid.png',
{ maxDiffPixelRatio: 0.02 } ); }); test('Full Haven Park page –
canvas masked', async ({ page }) => {      await page.goto('/lobby');
await page.locator('[data-testid=room-haven-park]').click();      await
page.waitForFunction(() => (window as any).__havenRoomReady === true);
await page.waitForTimeout(1500);      await
expect(page).toHaveScreenshot('haven-park-full-page.png',
{      maxDiffPixelRatio: 0.05,          // Mask the game canvas – only test
overlays and chrome      mask: [page.locator('canvas[data-testid=game-
canvas]')],      }); }); });

```

Tip: Updating Baselines

When you intentionally change the UI, update the baseline snapshots with: `npx playwright test e2e/visual/ --update-snapshots`. Always commit the updated .png snapshot files to git so CI has the correct baseline. Snapshots live in `e2e/visual/__screenshots__`.

16. Performance Testing — Playwright CDP MAC

Performance tests use Playwright's access to the Chrome DevTools Protocol (CDP) to measure real metrics: heap memory, FPS from Babylon.js's own engine counter, and page load timing. These tests are slow (5–8 minutes total) and are NOT run in the default CI pipeline — only on a nightly schedule or manually before a release. The thresholds chosen here reflect the minimum acceptable experience on a mid-range laptop.

File: e2e/performance/perf.spec.ts

```
// e2e/performance/perf.spec.ts import { test, expect, chromium } from
'@playwright/test'; // Performance tests: Chromium only (CDP not available
in Firefox/WebKit) test.use({ browserName: 'chromium' });
test.describe('Performance Benchmarks', () => { // — Test 16a
  test('Initial page
load time < 3 seconds', async ({ page }) => { const startTime =
Date.now(); await page.goto('/', { waitUntil: 'networkidle' }); const
loadTime = Date.now() - startTime; console.log(`[PERF] Initial load: $
${loadTime}ms`); expect(loadTime).toBeLessThan(3000); }); // — Test
16b
  test('Room
load time (click → canvas stable) < 5 seconds', async ({ page }) =>
{ test.use({ storageState: 'e2e/.auth/user_a.json' }); await
page.goto('/lobby'); const roomLoadStart = Date.now(); await
page.locator('[data-testid=room-haven-park]').click(); await
page.waitForFunction(() => (window as any).__havenRoomReady === true, {
timeout: 10_000, }); const roomLoadTime = Date.now() - roomLoadStart;
console.log(`[PERF] Room load: ${roomLoadTime}ms`);
expect(roomLoadTime).toBeLessThan(5000); }); // — Test 16c
  test('Memory heap
after 5 minutes in room < 150MB', async ({ browser }) => { const context
= await browser.newContext({ storageState: 'e2e/.auth/user_a.json' });
const page = await context.newPage(); // Open CDP session for memory
measurement const cdpSession = await context.newCDPSession(page);
await cdpSession.send('Performance.enable'); await page.goto('/lobby');
await page.locator('[data-testid=room-haven-park]').click(); await
page.waitForFunction(() => (window as any).__havenRoomReady ===
true); // Simulate 5 minutes of idle presence in the room await
page.waitForTimeout(5 * 60 * 1000); const metrics = await
cdpSession.send('Performance.getMetrics'); const heapMetric =
metrics.metrics.find((m: any) => m.name === 'JSHeapUsedSize'); const
heapMB = (heapMetric?.value ?? 0) / 1024 / 1024; console.log(`[PERF]
Heap after 5 min: ${heapMB.toFixed(1)}MB`);
expect(heapMB).toBeLessThan(150); await context.close(); }); // —
Test 16d
  test('FPS
during avatar movement: p95 > 30 FPS', async ({ page }) => { await
page.goto('/lobby'); await page.locator('[data-testid=room-haven-
park]').click(); await page.waitForFunction(() => (window as
any).__havenRoomReady === true); const fpsSamples: number[] =
[]; // Sample FPS 20 times over 10 seconds while moving the avatar
for (let i = 0; i < 20; i++) { await page.locator('canvas[data-
testid=game-canvas]').click({ position: { x: 200 + (i * 5), y: 300 },
}); const fps = await page.evaluate(() => (window as
any).__havenEngine?.getFps() ?? 0 ); fpsSamples.push(fps);
await page.waitForTimeout(500); } // Calculate p95: sort descending,
take index at 5th percentile from top fpsSamples.sort((a, b) => a - b);
const p95Index = Math.floor(fpsSamples.length * 0.05); const p95Fps =
fpsSamples[p95Index]; console.log(`[PERF] FPS samples: [$
{fpsSamples.join(', ')}] → p95: ${p95Fps}`);
expect(p95Fps).toBeGreaterThan(30); }); // — Test 16e
  test('No memory
leak: heap delta before/after room load within 10MB', async ({ browser }) =>
{ const context = await browser.newContext({ storageState:
'e2e/.auth/user_a.json' }); const page = await context.newPage();
const cdpSession = await context.newCDPSession(page); await
cdpSession.send('Performance.enable'); await page.goto('/lobby');
await page.evaluate(() => window.gc?.()); // force GC before baseline
await page.waitForTimeout(500); const before = await
```

```

cdpSession.send('Performance.getMetrics');    const heapBefore =
before.metrics.find((m: any) => m.name === 'JSHeapUsedSize')?.value ?? 0;
// Load a room, stay 30s, then unload    await page.locator('[data-
testid=room-haven-park]').click();    await page.waitForFunction(() =>
(window as any).__havenRoomReady === true);    await
page.waitForTimeout(30_000);    await page.locator('[data-testid=leave-room-
btn]').click();    await page.waitForURL('**/lobby');    // Force GC and
measure    await page.evaluate(() => window.gc?.());    await
page.waitForTimeout(500);    const after = await
cdpSession.send('Performance.getMetrics');    const heapAfter =
after.metrics.find((m: any) => m.name === 'JSHeapUsedSize')?.value ?? 0;
const deltaMB = (heapAfter - heapBefore) / 1024 / 1024;
console.log(`[PERF] Heap delta after room load/unload: $
{deltaMB.toFixed(2)}MB`);    expect(Math.abs(deltaMB)).toBeLessThan(10);
await context.close();    }); });

```

Note: Run Performance Tests in Isolation

Add `--project=chromium` and run performance tests alone: `npm run test:perf`.
Never run performance tests in parallel with other tests — shared CPU time
distorts FPS and memory measurements. On CI, schedule them as a nightly cron
job separate from the PR test suite.

17. Accessibility Testing MAC

Accessibility testing is not optional for a social platform. Players with motor disabilities rely on keyboard navigation; players with visual impairments may use screen readers for the HTML overlay UI. The Babylon.js canvas itself cannot be made accessible, but all overlays — chat, login, shop, avatar customizer — must meet WCAG 2.1 Level AA. These Playwright tests validate keyboard flow and ARIA attribute presence.

File: `e2e/accessibility/a11y.spec.ts`

```

// e2e/accessibility/a11y.spec.ts import { test, expect } from
'@playwright/test'; test.describe('Accessibility – Keyboard Navigation', ()
=> {    test('(a) Login form: Tab order correct, Enter submits', async
({ page }) => {    await page.goto('/login');    // Tab should move focus
in this order: email → password → submit button    await
page.keyboard.press('Tab');    await
expect(page.locator('[name=email]').toBeFocused();    await
page.keyboard.press('Tab');    await
expect(page.locator('[name=password]').toBeFocused();    await
page.keyboard.press('Tab');    await

```

```

expect(page.locator('button[type=submit]')).toBeFocused(); // Fill
fields and submit via Enter key await
page.locator('[name=email]').fill('testuser_a@havenworld.test'); await
page.locator('[name=password]').fill(process.env.TEST_USER_A_PASS!);
await page.keyboard.press('Enter'); await
expect(page).toHaveURL(/\/lobby/, { timeout: 8_000 }); }); test('(b)
Chat overlay: Tab to input, Enter sends message', async ({ page }) =>
{ test.use({ storageState: 'e2e/.auth/user_a.json' }); await
page.goto('/lobby'); await page.locator('[data-testid=room-haven-
park]').click(); await page.waitForFunction(() => (window as
any).__havenRoomReady === true); // Press T to focus chat (game
convention) or Tab to reach it await page.keyboard.press('t'); await
expect(page.locator('[aria-label="Chat message"]')).toBeFocused(); await
page.keyboard.type('Keyboard test message'); await
page.keyboard.press('Enter'); await expect(page.locator('[data-
testid=chat-messages]')).toContainText('Keyboard test message',
{ timeout: 5_000 }); }); test('(c) Shop modal: Tab through items,
Space selects, Escape closes', async ({ page }) => { await
page.goto('/lobby'); await page.locator('[data-testid=open-shop-
btn]').click(); await expect(page.locator('[data-testid=shop-
modal]')).toBeVisible(); // First focusable element in modal should be
focused (focus trap) await page.keyboard.press('Tab'); await
expect(page.locator('[data-testid=shop-item]:first-child')).toBeFocused();
// Space to select item await page.keyboard.press('Space'); await
expect(page.locator('[data-testid=item-detail-
panel]')).toBeVisible(); // Escape to close modal entirely await
page.keyboard.press('Escape'); await expect(page.locator('[data-
testid=shop-modal]')).not.toBeVisible(); }); });
test.describe('Accessibility – ARIA Labels', () => { test('(a) Chat input
has aria-label="Chat message"', async ({ page }) => { await
page.goto('/lobby'); await page.locator('[data-testid=room-haven-
park]').click(); await page.waitForFunction(() => (window as
any).__havenRoomReady === true); const chatInput = page.locator('[aria-
label="Chat message"]'); await expect(chatInput).toBeVisible(); await
expect(chatInput).toHaveAttribute('aria-label', 'Chat message'); });
test('(b) Avatar customizer sliders have correct ARIA value attributes',
async ({ page }) => { await page.goto('/avatar-customizer'); const
slider = page.locator('[data-testid=body-type-slider]'); await
expect(slider).toHaveAttribute('aria-valuemin', '0'); await
expect(slider).toHaveAttribute('aria-valuemax', '1'); await
expect(slider).toHaveAttribute('aria-valuenow'); await
expect(slider).toHaveAttribute('aria-label'); }); test('(c) Shop items
have aria-label with name and price', async ({ page }) => { await
page.goto('/lobby'); await page.locator('[data-testid=open-shop-
btn]').click(); const firstItem = page.locator('[data-testid=shop-
item]:first-child'); const ariaLabel = await
firstItem.getAttribute('aria-label'); // aria-label should include item
name AND price, e.g. "Oak Chair – 250 coins" expect(ariaLabel).toMatch(/.
+-.\d+/); }); });

```

18. Mobile Emulation Testing MAC

Playwright's device emulation sets viewport size, device pixel ratio, user-agent string, and touch event support simultaneously — replicating a real mobile device

without requiring a physical phone. HavenWorld uses touch-to-move for avatar control on mobile (tap the ground → avatar walks there), which is a fundamentally different input mode from desktop mouse-click movement. Test both separately.

File: e2e/mobile/mobile.spec.ts

```
// e2e/mobile/mobile.spec.ts import { test, expect, devices } from
'@playwright/test'; test.describe('Mobile – iPhone 14', () => { test.use({
...devices['iPhone 14'], storageState: 'e2e/.auth/user_a.json' });
test('(a) Login form renders on 375px viewport – no horizontal scroll', async
({ page }) => { await page.goto('/login'); // Confirm viewport width
is 375px (iPhone 14 logical pixels) const width = await page.evaluate(()
=> document.body.scrollWidth); expect(width).toBeLessThanOrEqual(375);
await expect(page.locator('[name=email]')).toBeVisible(); await
expect(page.locator('[name=password]')).toBeVisible(); await
expect(page.locator('button[type=submit]')).toBeVisible(); }); test('(b)
Chat overlay toggles with touch button', async ({ page }) => { await
page.goto('/lobby'); await page.locator('[data-testid=room-haven-
park]').click(); await page.waitForFunction(() => (window as
any).__havenRoomReady === true); const chatToggle = page.locator('[data-
testid=chat-toggle]'); const chatOverlay = page.locator('[data-
testid=chat-overlay]'); // Chat is hidden by default on mobile (canvas
space is precious) await expect(chatOverlay).not.toBeVisible();
await chatToggle.tap(); await expect(chatOverlay).toBeVisible();
await chatToggle.tap(); await
expect(chatOverlay).not.toBeVisible(); }); test('(c) Avatar movement via
tap on canvas', async ({ page }) => { await page.goto('/lobby');
await page.locator('[data-testid=room-haven-park]').click(); await
page.waitForFunction(() => (window as any).__havenRoomReady ===
true); // Tap the canvas to trigger touch-based pathfinding move
const movedPromise = page.evaluate(() => new Promise<boolean>(resolve => {
(window as any).__havenSocket.once('PLAYER_MOVED', () =>
resolve(true)); })); await page.locator('canvas[data-testid=game-
canvas]').tap({ position: { x: 187, y: 300 } }); const moved = await
movedPromise; expect(moved).toBe(true); }); test('(d) Shop modal is
scrollable on 375px screen', async ({ page }) => { await
page.goto('/lobby'); await page.locator('[data-testid=open-shop-
btn]').click(); await expect(page.locator('[data-testid=shop-
modal]')).toBeVisible(); // The modal should not overflow the viewport
(no clipped content) const modal = page.locator('[data-testid=shop-
modal]'); const box = await modal.boundingBox();
expect(box!.x).toBeGreaterThanOrEqual(0); // Scroll down inside modal to
confirm it's scrollable await modal.evaluate(el => el.scrollTop = 200);
const scrollTop = await modal.evaluate(el => el.scrollTop);
expect(scrollTop).toBeGreaterThanOrEqual(0); }); test('(e) Canvas fills
viewport, no horizontal scroll', async ({ page }) => { await
page.goto('/lobby'); await page.locator('[data-testid=room-haven-
park]').click(); await page.waitForFunction(() => (window as
any).__havenRoomReady === true); const canvas =
page.locator('canvas[data-testid=game-canvas]'); const box = await
canvas.boundingBox(); // Canvas width should fill the full 375px
viewport expect(box!.width).toBeCloseTo(375, 5); // Page body should
not have horizontal scroll const bodyScroll = await page.evaluate(() =>
document.body.scrollWidth > window.innerWidth);
expect(bodyScroll).toBe(false); }); }); test.describe('Mobile – Samsung
Galaxy S23', () => { test.use({ ...devices['Galaxy S23'], storageState:
'e2e/.auth/user_a.json' }); test('Login form renders on Galaxy S23
viewport', async ({ page }) => { await page.goto('/login'); await
```

```
expect(page.locator('[name=email]').toBeVisible());    await
expect(page.locator('button[type=submit]').toBeVisible());    const width =
await page.evaluate(() => document.body.scrollWidth);
expect(width).toBeLessThanOrEqual(412);    }); }); test.describe('Mobile –
iPad Pro', () => {    test.use({ ...devices['iPad Pro 11'], storageState:
'e2e/.auth/user_a.json' });    test('Room loads and fills iPad viewport
correctly', async ({ page }) => {    await page.goto('/lobby');    await
page.locator('[data-testid=room-haven-park]').click();    await
page.waitForFunction(() => (window as any).__havenRoomReady === true);
const canvas = page.locator('canvas[data-testid=game-canvas]');    const box
= await canvas.boundingBox();    // iPad Pro 11" = 834px logical width
expect(box!.width).toBeCloseTo(834, 10);    }); });
```

19. Test Data Management

19.1 Playwright storageState — Authenticated Sessions

Re-logging in at the start of every E2E test wastes 2–4 seconds per test and creates flakiness if the auth server is slow. Playwright's `storageState` lets you save all cookies and localStorage after a single login, then restore that state instantly in any subsequent test context. This is the correct pattern for any test that requires authentication but is not testing the login flow itself.

```
// e2e/globalSetup.ts – storageState is saved here once, reused by all tests
// Store auth state for two test users in version-controlled files //
(gitignore the actual files – they contain session tokens) const
AUTH_STATE_DIR = 'e2e/.auth/'; await fs.mkdir(AUTH_STATE_DIR, { recursive:
true }); // Save User A session await
loginAndSave('testuser_a@havenworld.test', process.env.TEST_USER_A_PASS!,
'user_a.json'); // Save User B session (for multiplayer tests) await
loginAndSave('testuser_b@havenworld.test', process.env.TEST_USER_B_PASS!,
'user_b.json'); async function loginAndSave(email: string, password: string,
filename: string) {    const browser = await chromium.launch();    const page =
await browser.newPage();    await page.goto('http://localhost:5173/login');
await page.fill('[name=email]', email);    await page.fill('[name=password]',
password);    await page.click('button[type=submit]');    await
page.waitForURL('**/lobby');    await page.context().storageState({ path:
path.join(AUTH_STATE_DIR, filename) });    await browser.close(); }
```

19.2 Environment Variables

```
# .env.test – test environment config (committed without secrets)
VITE_API_URL=http://localhost:4000 VITE_SOCKET_URL=http://localhost:4000
VITE_ENV=test TEST_INVITE_CODE=HAVEN-TEST-2026
TEST_API_URL=http://localhost:4000 # .env.test.local – secrets (gitignored,
never committed) TEST_USER_A_PASS=SecureTestPass_A!2026
TEST_USER_B_PASS=SecureTestPass_B!2026
```

19.3 beforeEach Room State Reset

```
// Reusable fixture – call in any test that needs a clean room state async
function resetRoomState(roomId: string) {  const res = await fetch(`$
{process.env.TEST_API_URL}/api/test/rooms/${roomId}/reset`, { method:
'POST', headers: { 'x-test-key': process.env.TEST_API_KEY! } } );  if (!
res.ok) throw new Error(`Failed to reset room ${roomId}: $
{res.status}`); } // Usage in a test file test.beforeEach(async () =>
{  await resetRoomState('haven-park'); });
```

19.4 Tear-Down After E2E Suite

```
// e2e/globalTeardown.ts export default async function globalTeardown()
{  // Delete all test user data created during this run  await fetch(`$
{process.env.TEST_API_URL}/api/test/cleanup`, {  method: 'DELETE',
headers: {  'Content-Type': 'application/json',  'x-test-key':
process.env.TEST_API_KEY!,  },  body: JSON.stringify({  users:
['testuser_a@havenworld.test', 'testuser_b@havenworld.test'],  })),  });
// Stop Vite dev server and backend (started in globalSetup)  await
viteServer?.close();  backendProcess?.kill('SIGTERM');  console.log('[E2E
Teardown] Complete'); }
```

Tip: Test API Endpoints Are Test-Mode Only

The `/api/test/` routes (`/cleanup`, `/reset`, `/verify-email`) must only exist when `NODE_ENV=test`. Guard them with a middleware check: `if (process.env.NODE_ENV !== 'test') return res.status(404).end();`. If these routes accidentally reach production, they are a critical security vulnerability.

20. CI Integration — GitHub Actions

MAC

The client test suite is split into two GitHub Actions workflows: a fast unit/component/scene workflow that runs on every PR, and a slower E2E workflow that runs only on PRs targeting `main`. Performance and mobile tests run on a nightly schedule. This split keeps the inner-loop feedback fast (under 90 seconds for unit tests) while still catching E2E regressions before they reach production.

20.1 client-test.yml — Unit Tests (Every PR)

```
# .github/workflows/client-test.yml name: Client Unit Tests on:
pull_request:      branches: ['*'] push:      branches: [main] jobs:
vite:      name: Vitest — Unit / Component / Scene runs-on: ubuntu-
latest      timeout-minutes: 10      steps:      - uses: actions/checkout@v4
- name: Setup Node.js 22      uses: actions/setup-node@v4      with:
node-version: '22'      cache: 'npm'      - name: Install dependencies
run: npm ci      - name: Run client unit tests with coverage      run:
npm run test:client:coverage      env:      VITE_API_URL:
http://localhost:4000      VITE_ENV: test      - name: Upload coverage
report      uses: actions/upload-artifact@v4      if: always()
with:      name: client-coverage-${{ github.run_id }}      path:
coverage/client/      retention-days: 14
```

20.2 e2e.yml — Playwright E2E (PRs to main only)

```
# .github/workflows/e2e.yml name: Playwright E2E Tests on: pull_request:
branches: [main] jobs: playwright:      name: Playwright E2E — $
${{ matrix.browser }} runs-on: ubuntu-latest      timeout-minutes: 20
strategy:      fail-fast: false      matrix:      browser: [chromium,
firefox, webkit]      steps:      - uses: actions/checkout@v4      - name:
Setup Node.js 22      uses: actions/setup-node@v4      with:
node-version: '22'      cache: 'npm'      - name: Install dependencies
run: npm ci      - name: Install Playwright browsers      run: npx
playwright install --with-deps ${{ matrix.browser }}      - name: Build
server (needed for E2E backend)      run: npm run build:server      -
name: Run Playwright E2E tests      run: npx playwright test --project=${{
matrix.browser }}      env:      CI: true      TEST_API_URL:
http://localhost:4000      TEST_INVITE_CODE: $
${{ secrets.TEST_INVITE_CODE }}      TEST_USER_A_PASS: $
${{ secrets.TEST_USER_A_PASS }}      TEST_USER_B_PASS: $
${{ secrets.TEST_USER_B_PASS }}      TEST_API_KEY: $
${{ secrets.TEST_API_KEY }}      # Upload HTML report (screenshots + traces)
on failure for debugging      - name: Upload Playwright report      uses:
actions/upload-artifact@v4      if: failure()      with:
name: playwright-report-${{ matrix.browser }}-${{ github.run_id }}
path: playwright-report/      retention-days: 7
```

20.3 nightly.yml — Performance + Mobile (Nightly Schedule)

```
# .github/workflows/nightly.yml name: Nightly — Performance & Mobile Tests
on:      schedule:      # 3:00 AM UTC (10:00 PM CDT) nightly      - cron: '0 3 * *
*'      workflow_dispatch: # Allow manual trigger jobs:      performance:
name: Performance Benchmarks runs-on: ubuntu-latest      timeout-minutes:
30      steps:      - uses: actions/checkout@v4      - uses: actions/setup-
node@v4      with: { node-version: '22', cache: 'npm' }      - run: npm
ci      - run: npx playwright install --with-deps chromium      - run: npm
run build:server      - run: npm run test:perf      env:      CI:
true      TEST_USER_A_PASS: ${{ secrets.TEST_USER_A_PASS }}
TEST_API_KEY: ${{ secrets.TEST_API_KEY }}      - uses: actions/upload-
artifact@v4      if: always()      with:      name: perf-report-
${{ github.run_id }}      path: playwright-report/      mobile:      name:
Mobile Emulation Tests runs-on: ubuntu-latest      timeout-minutes: 25
```

```

steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-node@v4
  with: { node-version: '22', cache: 'npm' }
  - run: npm ci
  - run:
    npx playwright install --with-deps chromium webkit
    build:server
    - run: npm run test:mobile
    env:
      CI: true
TEST_USER_A_PASS: ${ secrets.TEST_USER_A_PASS }
TEST_API_KEY: ${ secrets.TEST_API_KEY }

```

💡 Tip: Playwright Report as CI Artifact

When Playwright E2E tests fail in CI, the HTML report artifact is the fastest path to diagnosis. It includes: a list of all failed tests, full error messages, browser screenshots at the moment of failure, and recorded video of the failing test session. Download the artifact from the GitHub Actions run summary page: Actions → <run> → Artifacts → playwright-report-chromium-<run_id> → npx playwright show-report.

21. Client Test Coverage Checklist

Use this table as a quick reference for test coverage status. Update the Status column as tests are implemented. Target: all items **Done** before the Part 8 CI/CD hardening pass.

Service / Component	Test Type	Tool	File Path	Test Cases	Status
AuthService	Unit	Vitest	src/client/services/__tests__/authService.test.ts	7 (login, logout, token expiry, XSS guard)	✅ Done
SocketService	Unit	Vitest + vi.mock	src/client/services/__tests__/socketService.test.ts	7 (connect, queue, backoff, timeout)	✅ Done
AvatarController	Unit (Scene)	Vitest + NullEngine	src/client/game/__tests__/avatarController.test.ts	8 (morph targets, animations, navmesh)	✅ Done

Service / Component	Test Type	Tool	File Path	Test Cases	Status
RoomLoader	Unit (Scene)	Vitest + NullEngine	src/client/game/__tests__/roomLoader.test.ts	6 (GLB load, navmesh, unload, memory)	✅ Done
Furniture Manager	Unit (Scene)	Vitest + NullEngine	src/client/game/__tests__/furnitureManager.test.ts	7 (place, remove, move, limit, serialize)	✅ Done
AudioEngine	Unit	Vitest + vi.mock	src/client/game/__tests__/audioEngine.test.ts	8 (synth sounds, mute, unknown SFX)	✅ Done
ChatOverlay	Component	Vitest + jsdom + @testing-library/dom	src/client/ui/__tests__/chatOverlay.test.ts	8 (render, submit, XSS, rate limit, history)	✅ Done
RoomScene (NullEngine)	Scene	Vitest + NullEngine	__tests__/scene/roomScene.test.ts	9 (lighting, fog, physics, pet FSM, navmesh)	✅ Done
Full Game Flow	E2E	Playwright (3 browsers)	e2e/fullFlow.spec.ts	7 steps: register → login → room → move → chat → logout	✅ Done
Two-Player Multiplayer	E2E	Playwright (dual context)	e2e/multiplayer.spec.ts	5 (movement sync, chat relay, P2P trade)	✅ Done
Avatar Customization	E2E	Playwright	e2e/avatarCustomizer.spec.ts	2 (save+reload persist, live preview)	✅ Done
Visual Regression	E2E (Screenshot)	Playwright (Chromium only)	e2e/visual/rooms.spec.ts	4 (chat overlay, customizer, shop modal, full page masked)	✅ Done
Performance Benchmark	E2E (CDP)	Playwright + CDP (Chromium)	e2e/performance/perf.spec.ts	5 (load time, room time, heap, FPS)	✅ Done

Service / Component	Test Type	Tool	File Path	Test Cases	Status
ks				p95, leak)	
Accessibility	E2E	Playwright	e2e/accessibility/a11y.spec.ts	6 (keyboard nav, ARIA labels, focus trap)	✅ Done
Mobile Emulation	E2E (Device)	Playwright (device profiles)	e2e/mobile/mobile.spec.ts	7 (iPhone 14, Galaxy S23, iPad Pro)	✅ Done
ShopModal	Component	Vitest + jsdom	src/client/ui/__tests__/shopModal.test.ts	TBD (render, item select, purchase confirm)	❑ Planned
LoginForm	Component	Vitest + jsdom	src/client/ui/__tests__/loginForm.test.ts	TBD (validation, error states, submit)	❑ Planned
Inventory Panel	Component	Vitest + jsdom	src/client/ui/__tests__/inventoryPanel.test.ts	TBD (render items, drag-and-drop to room)	❑ Planned
PetManager	Unit (Scene)	Vitest + NullEngine	src/client/game/__tests__/petManager.test.ts	TBD (spawn, FSM states, multiple pets)	❑ Planned
TradeFlow E2E	E2E	Playwright (dual context)	e2e/trade.spec.ts	TBD (full trade flow separate from multiplayer spec)	❑ Planned

Summary — Part 7B Complete

This guide covered the complete HavenWorld client testing stack: **Vitest 2.x** for unit, component, and NullEngine scene tests; **Playwright 1.50+** for E2E, multiplayer, visual regression, performance CDP, accessibility, and mobile emulation; and GitHub Actions workflows to run them all automatically. Estimated coverage baseline at completion: **~62% lines covered** (all 6

services + 2 UI components unit-tested), with E2E covering all critical user paths. Part 8 will build on this foundation with deployment hardening, Cloudflare Pages preview environments, and branch-specific test gating.

HavenWorld Build Guide — Part 7B of 10 | Author: Daniel | 18 September 2026 |
Anoka, MN | All commands: [MAC](#) MacBook Pro unless tagged otherwise |
Deployment: <https://havenworld.pages.dev> | Free-tier infrastructure only